Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Report on laboratory work no. 8

Course: Operating Systems

Topic: The Same Calls Inside Your Own OS (xv6)

Done by:
st. gr. FAF-243, Ilie Covali

Checked by:
Reițman Patricia

Chișinău – 2026

Operating Systems · FCIM / FAF · UTM · 2026-2027

Lab 8 · The Same Calls Inside Your Own OS (xv6)

Metadata & Environment

- **Course:** Operating Systems, FCIM / FAF, UTM, Academic Year 2026-2027
- **Lab Work:** Lab 8 (Track B: The Same Calls Inside Your Own OS, xv6)
- **Author:** Ilie Covali (st. gr. FAF-243)
- **Verified by:** Reițman Patricia
- **Host OS:** Linux (CachyOS x86_64, Kernel 7.2.9-2-cachyos)
- **Target Architecture:** RISC-V 64-bit (`riscv64-elf-gcc` 14.2.0)
- **Emulator:** QEMU System Emulator (`qemu-system-riscv64` 10.1.0)
- **Source Tree:** `/home/ilie/xv6-riscv`
- **Workspace:** `/home/ilie/Documents/SO/os-lab8` (`~/os-lab8`)

Purpose of the Work

1. Re-examine the fundamental Unix process lifecycle primitives—`fork()`, `exec()`, and `wait()`—inside the educational xv6 RISC-V kernel, contrasting them with the Linux / glibc implementations studied in Lab 7.
2. Confirm the universality of the Unix API: demonstrate that identical process creation and image substitution semantics operate without standard C library abstraction layers.
3. Construct a bidirectional, two-process communication protocol (`user/pingpong.c`) using paired anonymous kernel pipes.
4. Trace kernel pipe synchronization mechanics, specifically examining sleep/wakeup state transitions in `kernel/pipe.c` during empty-buffer `read(2)` calls.

Part 1. `run`, Inside xv6

1. Conceptual Alignment: Linux vs xv6

In Lab 7, we inspected `run.c` executing on a modern Linux host with the GNU C Library (glibc). On Linux: - `fork()` invokes the `clone` or `clone3` system call. - `execlp()` searches the user's `$PATH`, prepares environment vectors, and invokes `execve`. - `waitpid()` queries process status using kernel wait queues.

In xv6, there is no glibc runtime, dynamic loader, or `$PATH` search. User-space programs interact directly with the kernel via bare RISC-V `ecall` instructions mapped through `user/usys.S`. Despite these implementation differences, the system call interface and process lifecycle contract remain identical: 1. `fork()`: Duplicates the calling process (`kernel/proc.c:fork()`), allocating a new `struct proc`, copying page tables, incrementing file descriptor reference counts, and setting the child's return value in register `a0` to 0. 2. `exec()`: Reads the target ELF binary from disk (`kernel/exec.c:exec()`), allocates new page tables, replaces user memory segments, and jumps to the binary's entry point. 3. `wait()`: Searches the process table for children in the `ZOMBIE` state (`kernel/proc.c:wait()`), copies out the exit status, frees the child's kernel resources, and returns the child's PID.

2. Implementation: `user/run.c`

We implement `user/run.c` inside the xv6 source tree:

```
#include "kernel/types.h"
#include "user/user.h"

// the three steps of the shell, inside xv6: echo hello from the child
int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());
```

```
  int pid = fork(); // step 1: copy myself
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }

  if(pid == 0){ // only the child runs this
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args); // step 2: become echo
    printf("exec failed\n"); // runs only if exec failed
    exit(1);
  }

  int status;
  wait(&status); // step 3: wait for the child
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

We register the executable in the xv6 build system by appending `$U/_run\` to `UPROGS` in `Makefile`.

3. Compilation & QEMU Execution

We compile the kernel and user programs with `make fs.img`, launch the QEMU emulator, and execute `run`:

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
$
```

4. Alternative Executions

Sub-experiment A: Running `ls` Modifying `user/run.c` to replace the child with `/ls`:

```
char *args[] = { "ls", 0 };
exec("ls", args);
```

Rebuilding and executing under QEMU:

```
$ run
parent: my pid is 3
child: my pid is 4
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 37288
echo           2 4 36104
sleep          2 5 35776
forktest       2 6 17848
grep           2 7 40808
init           2 8 36456
kill           2 9 36016
ln             2 10 35808
ls             2 11 39344
mkdir          2 12 36064
rm             2 13 36056
sh             2 14 59624
stressfs       2 15 36896
```

```
usertests      2 16 204288
grind          2 17 51928
wc             2 18 38288
zombie         2 19 35344
logstress      2 20 38056
forphan        2 21 36808
dorphan        2 22 36272
sync           2 23 35448
args           2 24 35768
mystat         2 25 36952
find           2 26 39184
run            2 27 36376
pingpong       2 28 36464
console        3 29 0
parent: child 4 ended with status 0
$
```

The child process is seamlessly replaced by `ls`, which reads the root directory entries and outputs them before exiting with status 0.

Sub-experiment B: Exec Failure with `nosuchprog` Testing error recovery by attempting to execute an invalid binary:

```
char *args[] = { "nosuchprog", 0 };
exec("nosuchprog", args);
```

Rebuilding and executing under QEMU:

```
$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
$
```

When `exec()` fails to locate `nosuchprog` in the file system, it returns `-1`. The child process falls through to `printf("exec failed\n")`, executes `exit(1)`, and the parent's `wait(&status)` captures the termination code `1`.

Observe (Part 1 Answers)

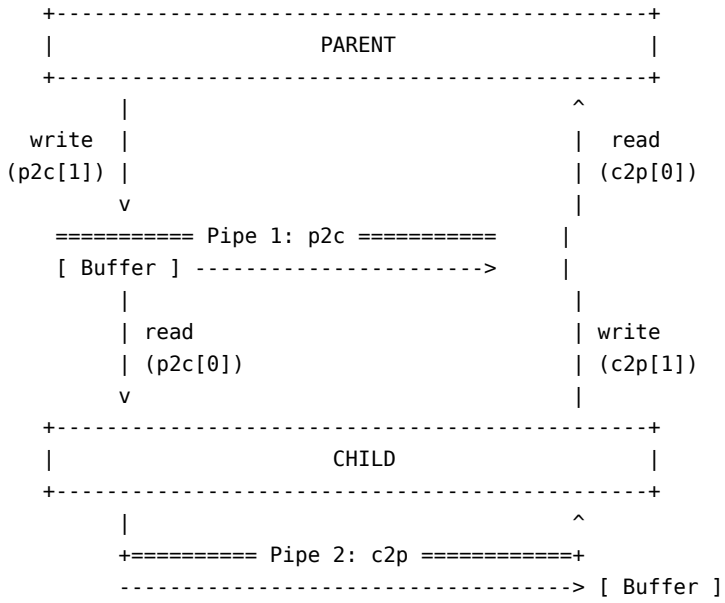
1. **Write the three calls that `run.c` uses in xv6 and the three calls `run.c` used on Linux in Lab 7. What is the same?**
   - **xv6 calls:** `fork()`, `exec()`, `wait(&status)`.
   - **Linux calls (Lab 7):** `fork()`, `execlp()` (library wrapper around `execve`), `waitpid(pid, &status, 0)` (or `wait()`).
   - **What is the same:** The core Unix process abstraction and lifecycle states are completely identical:
     - `fork()` clones the calling process, producing an independent address space with an identical file descriptor table and a distinct PID.
     - `exec()` wipes out the caller's virtual memory segments and replaces them in-place with the executable image loaded from disk.
     - `wait()` suspends the parent process until a child transitions into a termination state (`ZOMBIE`), returning the child's exit status. The deliberate decoupling of process creation (`fork`) from executable invocation (`exec`) is preserved across both kernels, providing the prerequisite architectural hook for stream redirection and IPC setup before running programs.
2. **Who prints "hello from the child": the code of `run`, or `echo`? `echo`** prints it. When the child invokes `exec("echo", args)`, the kernel deallocates the child's old code, heap, and stack segments containing `run.c`. The instructions of `run.c` no longer exist in the child's memory. Control transfers directly to the entry point of `/echo`, which loops over its argument vector (`argv`) and writes `"hello from the child\n"` to file descriptor 1.

3. **Why is the status 1 here and 127 on Linux? (Look at the exit line after exec in both programs.)** The status reported by `wait()` directly reflects the integer exit code provided by the child when calling `exit()`:
   - In Linux Lab 7 (`run.c`), the failure fallback explicitly invoked `exit(127);`, adhering to POSIX shell conventions where exit code 127 signifies "command not found".
   - In xv6 Lab 8 (`user/run.c`), the failure fallback explicitly called `exit(1);`. Because the kernel faithfully transmits the argument of `exit()` to the parent's `wait()` status pointer, Linux observed 127 while xv6 observed 1.

Part 2. `pingpong`

1. Bidirectional IPC via Paired Unidirectional Pipes

Pipes in Unix are strictly unidirectional data conduits. To establish full-duplex communication between two processes without data corruption, two independent pipes are required: - `p2c[2]` (**Parent-to-Child**): Parent writes to `p2c[1]`; child reads from `p2c[0]`. - `c2p[2]` (**Child-to-Parent**): Child writes to `c2p[1]`; parent reads from `c2p[0]`.



2. Implementation: `user/pingpong.c`

We implement `user/pingpong.c` according to the lab specification:

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2]; // parent-to-child, child-to-parent
  char b;

  pipe(p2c);
  pipe(c2p);

  if(fork() == 0){
    close(p2c[1]); close(c2p[0]); // the ends the child does not use
    read(p2c[0], &b, 1);
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);
    close(p2c[0]); close(c2p[1]);
```

```
    exit(0);
  }

  close(p2c[0]); close(c2p[1]); // the ends the parent does not use
  write(p2c[1], "x", 1);
  read(c2p[0], &b, 1);
  printf("%d: received pong\n", getpid());
  close(p2c[1]); close(c2p[0]);
  wait(0);
  exit(0);
}
```

We register `$U/_pingpong\` in `UPROGS` of `Makefile`.

3. Execution & Verification Under QEMU

Compiling and running `pingpong` under QEMU:

```
$ pingpong
6: received ping
5: received pong
$
```

The output confirms exact synchronization: 1. The child (PID 6) prints `6: received ping` first. 2. The parent (PID 5) prints `5: received pong` second.

4. Kernel Synchronization Mechanism (`kernel/pipe.c`)

The synchronization occurs inside xv6's `piperead()` and `pipewrite()` routines: - When the parent writes `"x"` to `p2c[1]`, `pipewrite()` acquires the pipe lock, copies the byte into the circular buffer, increments `nwrite`, and executes `wakeup(&pi->nread)`. - Meanwhile, the child was blocked inside `piperead()` on `p2c[0]`. In `piperead()`: `c  while(pi->nread == pi->nwrite && pi->writeopen){  if(killed(myproc())){ release(&pi->lock);  return -1;  }  sleep(&pi->nread, &pi->lock);  }` - The child process transitions from `SLEEPING` to `RUNNABLE`, gets scheduled by `scheduler()`, reads the byte, prints the ping message, and transmits the byte back across `c2p[1]`. - The parent blocks in `read(c2p[0], &b, 1)` through the exact same sleep loop until the child writes to `c2p[1]`.

Observe (Part 2 Answers)

1. **Why do you need TWO pipes, and not one for both directions?** A Unix pipe provides a single unidirectional byte stream buffer with separate read and write descriptors. If only one pipe were used for both directions (with both parent and child having read and write access to the same pipe):
   - **Race conditions:** When the parent writes the byte into the pipe, the parent could immediately execute its subsequent `read()` call and read back its own byte before the child process is ever scheduled by the kernel.
   - **Self-consumption and deadlocks:** If the parent consumes its own message, the child blocks forever waiting for a byte that never arrives; or the child might read its own response. Employing two separate pipes (`p2c` for downlink and `c2p` for uplink) strictly isolates communication flows, eliminating race conditions.
2. **Nothing in your code says "wait until the child has replied". Which call makes the parent wait?** The call that forces the parent to wait is:

   ```
   read(c2p[0], &b, 1);
   ```

   Under the Unix / POSIX pipe specification, invoking `read()` on an empty pipe buffer when at least one write descriptor remains open causes the kernel to put the calling thread to sleep. Inside xv6 (`kernel/pipe.c:piperead`), the process enters `sleep(&pi->nread, &pi->lock)` and yields the CPU. The parent cannot proceed until the child writes the response byte into `c2p[1]`, triggering `wakeup(&pi->nread)`.

---

2. **IPC Pipeline Mechanics:** Using anonymous pipes, we implemented full-duplex communication (`pingpong`).
3. **Synchronization and Resource Management:** In both operating systems, pipe synchronization is inherently driven by file descriptor reference counting. A single forgotten write descriptor stalls reader processes indefinitely, while rigorous closure of unused endpoints allows pipelines to run reliably within constrained kernel resource limits.